Shlok Bam

Test ID: 450016892655551 | 7974670370 | shlok.bam23@vit.edu

Test Date: March 14, 2026

Computer Science	Logical Ability	Node.js	Quantitative Ability (Advanced)
100 /100	73 /100	100 /100	83 /100

English Comprehension	ReactJS	Automata Fix	Automata Pro
87 /100	100 /100	72 /100	95 /100

Personality
Completed

Computer Science

100 / 100

OS and Computer Architecture	DBMS	Computer Networks
100 / 100	100 / 100	100 / 100

Logical Ability

73 / 100

Inductive Reasoning	Deductive Reasoning	Abductive Reasoning
76 / 100	74 / 100	69 / 100

Node.js

100 / 100

Basics and Events	File Systems and Error Handling	DB, Server and Testing
100 / 100	100 / 100	100 / 100

Quantitative Ability (Advanced)

83 / 100

Basic Mathematics	Advanced Mathematics	Applied Mathematics
85 / 100	80 / 100	83 / 100

© 2026 SHL and/or its affiliates. All rights reserved..

English Comprehension

87 / 100 | CEFR: **C2**

Grammar	Vocabulary	Comprehension
87 / 100	83 / 100	90 / 100

ReactJS

100 / 100

React Basics	Components and Styling	Forms and Routing
100 / 100	100 / 100	100 / 100

Automata Fix

72 / 100

Syntactical Error	Logical Error	Code Reuse
0 / 100	100 / 100	50 / 100

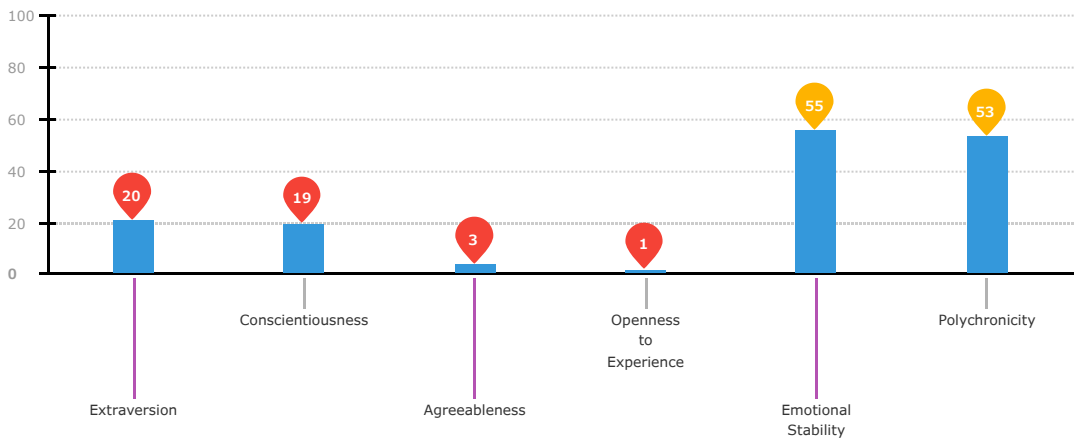
Automata Pro

95 / 100

Programming Ability	Programming Practices	Functional Correctness
100 / 100	75 / 100	100 / 100

Personality

Completed



Competencies

People Interaction

Self-Drive

Trainability

Repetitive Job Suitability

Work attributes

© 2026 SHL and/or its affiliates. All rights reserved..

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- 70 ≤ Score < 100
- 30 ≤ Score < 70
- 0 ≤ Score < 30

© 2026 SHL and/or its affiliates. All rights reserved..

2 Insights

English Comprehension

87 / 100 | CEFR: **C2**

This test aims to measure your vocabulary, grammar and reading comprehension skills.

You have a fairly rich vocabulary and a strong command of English grammar. You are able to read and understand complex text. Having a good command of the English language is important to communicate with internal stakeholders and clients, as well as to interpret reports, articles and complex texts at work.

Logical Ability

73 / 100

Inductive Reasoning

76 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.

Deductive Reasoning

74 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.

Abductive Reasoning

69 / 100

Quantitative Ability (Advanced)

83 / 100

This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

It is commendable that you are able to understand and solve complex arithmetic problems. You are able to solve basic problems of probability, logarithms, permutations, and combinations. This skill will help you in jobs where one needs to work with statistical data and make probabilistic predictions.

Personality

Competencies

© 2026 SHL and/or its affiliates. All rights reserved..

Extraversion

Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

- You feel comfortable spending time by yourself.
- You prefer spending time alone rather than in social gatherings.
- You may not enjoy activities that involve thrill and excitement.
- You are thoughtful, introspective and refrains from impulsive remarks/actions. You often keep your opinions and ideas to yourself
- You prefer to work on individual projects rather than group projects.
- You are more likely to prefer jobs that require minimal interaction with people.

Conscientiousness

Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You have an easy going attitude towards work.
- You tend to act spontaneously and may prefer to get started on a project or at work without laying out an organized plan.
- You tend to make quick decisions rather than spending time on reviewing the facts at hand.
- You have a flexible and spontaneous approach to work and is less likely to adhere to strict guidelines and policies.
- You sometimes feel insecure about your capabilities and could benefit from emotional support.
- You are likely to be content with your level of achievement and may not strive to achieve more.
- Your personality is more suited for jobs which demand fast results and are not as concerned with attention to detail.

© 2026 SHL and/or its affiliates. All rights reserved..

Agreeableness

Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are outspoken. You often play the role of a devil's advocate in discussions and question others' opinions and views.
- You are not gullible and are likely to carefully examine the situation before trusting something/someone.
- You may not be strongly affected by human suffering and may be perceived as indifferent.
- You are confident of your achievements and do not shy away from talking about them.
- You sometimes place self-interest above the needs of those around you. You are not willing to compromise your own views in order to accommodate the views of others.
- You are suitable for jobs that require tough objective decisions and hard negotiation.

Openness to Experience

Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You may not be very open to new experiences lying outside your comfort zone and tends to prefer routine over variety.
- You may be pragmatic and is likely to be conventional in your outlook and actions and may not pursue an experimental approach to problem-solving.
- You may not have an appreciation for art.
- You do not like to express your emotions and feelings to others.
- You tend to demonstrate concrete thinking with a focus on practical solutions, as opposed to abstract ideas.
- Your personality is more suited to job roles that require logical and rational thinking.

Emotional Stability

Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are calm and relaxed in most situations.
- You experience a range of emotions in high pressure situations. You tend to worry when working in critical conditions.
- You do not like attention drawn towards you. You take some time to become confident and comfortable around people.
- You subdue your impulses and tend to act in a rational manner.
- Your personality is suited for jobs that have a moderate amount of stress.

© 2026 SHL and/or its affiliates. All rights reserved..

Polychronicity

Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You neither have a strong preference nor dislike to perform multiple tasks simultaneously.
- You are open to both options - pursuing multiple tasks at the same time or working on a single project at a time.
- Whether or not you will succeed in a polychronous environment depends largely on your ability to do so.

Node.js

100 / 100

- Your knowledge of the different areas of Node.js is good.
- You have a sound theoretical as well as practical understanding of advanced concepts and algorithms.
- You have the required knowledge to design and develop complex problems efficiently.

ReactJS

100 / 100

© 2026 SHL and/or its affiliates. All rights reserved..

3 | Response

Automata Pro

95 / 100 Code Replay

Question 1 (Language: Python3.12)

A leader has a team with N members. They have assigned an error score to each member on their team based on the bugs that they have found in that particular team member's task. Because the error score has increased to a significantly large value, they want to give all the team members a chance to improve their error scores, thereby improving their reputation in the organization. They introduce a new rule that, whenever a team member completes a project successfully, the error score of that member decreases by a count P and the error score of all the other team members whose score is greater than zero decreases by a count Q.

Write an algorithm to help the team lead find the minimum number of projects that the team must complete in order to make the error score of all the team members zero.

Scores

Programming Ability

100 / 100

Completely correct. A correct implementation of the problem using the right control-structures and data dependencies.

Programming Practices

50 / 100

High readability, low on program structure. The source code contains redundant/improper coding constructs and a few readability and formatting issues.

Functional Correctness

100 / 100

Functionally correct source code. Passes all the test cases in the test suite for a given problem.

Final Code Submitted

Compilation Status: Pass

```python
import math

def solve():
    n = int(input())
    scores = list(map(int,input().split()))
    p = int(input())
    q = int(input())

    if all(s == 0 for s in scores):
        print(0)
        return

    def feasible(k):
        total_needed = 0
        for s in scores:
            if s == 0:
                continue
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: O(N logN)

*N represents number of team members

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

© 2026 SHL and/or its affiliates. All rights reserved..

```
18        base = k * q
19        if p>q:
20          if base >= s:
21            x = 0
22          else:
23            x = math.ceil((s-base) / (p-q))
24        elif p == q:
25          if base < s:
26            return False
27          x = 0
28        else:
29          if base < s:
30            return false
31          x = 0
32        total_needed += x
33        if total_needed > k:
34          return False
35      return True
36    lo, hi = 0, int(2e14)
37    while lo<hi:
38      mid = (lo+hi)//2
39      if feasible(mid):
40        hi = mid
41      else:
42        lo = mid+1
43    print(lo)
44  solve()
```

Readability & Language Best Practices

Line 4,6,7...: Variable name doesn't conform to snake_case naming style

Performance & Correctness

Line 4: local variable 'n' is assigned to but never used
Line 30: undefined name 'false'
Line 4: Unused variable 'n'

Test Case Execution

Passed TC: **100%**

Total score

12/12

100%	100%	100%
Basic(**10**/10)	Advance(**1**/1)	Edge(**1**/1)

Compilation Statistics

6	6	0	0	0	4
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:20:38**
Average time taken between two compile attempts:	**00:03:26**
Average test case pass percentage per compile:	**33.33%**

© 2026 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Python3.12)

The manager of a supermarket wants to hold an event at which they will distribute gift baskets to lucky customers. Each gift basket contains a pair of products. Each basket contains different product pairs but the overall value of the baskets may be the same. There are N types of products and each product has a price. The gift baskets will be awarded to the customers that pick a product pair that has a difference in price equal to the given integer value K.

Write an algorithm to help the manager find the total number of lucky customers who will win a gift basket.

Scores

Programming Ability

100 / 100

Completely correct. A correct implementation of the problem using the right control-structures and data dependencies.

Programming Practices

100 / 100

High readability, high on program structure. The source code is readable and does not consist of any significant redundant/improper coding constructs.

Functional Correctness

100 / 100

Functionally correct source code. Passes all the test cases in the test suite for a given problem.

Final Code Submitted — Compilation Status: Pass

```
from collections import Counter

n = int(input())
prices = list(map(int, input().split()))
k = int(input())
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

© 2026 SHL and/or its affiliates. All rights reserved..

```
6
7 freq = Counter(prices)
8 count = 0
9
10 for p in freq:
11     if k == 0:
12         count+= freq[p] * (freq[p]-1)//2
13     else:
14         if(p+k) in freq:
15             count += freq[p] * freq[p+k]
16 print(count)
```

Best case code: O(N)

*N represents number of different types of products

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

Readability & Language Best Practices

Line 14: Unnecessary parens after 'if' keyword
Line 3,4,7...: Constant name "n" doesn't conform to UPPER_CASE naming style

Test Case Execution

Passed TC: **100%**

Total score 17/17

100%	100%	100%
Basic(**8**/8)	Advance(**8**/8)	Edge(**1**/1)

Compilation Statistics

8	7	1	0	0	3
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:09:07**
Average time taken between two compile attempts:	**00:01:08**
Average test case pass percentage per compile:	**50.74%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2026 SHL and/or its affiliates. All rights reserved..

Automata Fix

72 / 100 Code Replay

Question 1 (Language: Java)

The function/method ***maxReplace*** prints space separated integers representing the input list after replacing all elements of the input list with the maximum element of the input list.

The function/method ***maxReplace*** accept two arguments - *size*, an integer representing the size of the input list and *inputList,* a list of integers representing the input list, respectively.

The function/method ***maxReplace*** compiles unsuccessfully due to compilation error. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
// You can print the values to stdout for debugging
class Solution
{
  void maxReplace(int size, int[] inputList)
  {
    int max =inputList[0];
    if(size>0)
    {
      for(int i=0;i<size;i++)
      {
        if(max<inputList[i])
        {
          max = inputList[i];
        }
      }
    }
    for(int i=0;i<size;i++)
    {
      inputList[i]=max;
      System.out.print(inputList[i]+ " ");
    }
  }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **87.5%**

Total score 7/8

100%	0%	0%
Basic(**7**/7)	Advance(**0**/0)	Edge(**0**/1)

© 2026 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

6	2	4	0	0	2
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:06:09**
Average time taken between two compile attempts:	**00:01:02**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

The function/method ***calculateMatrixSum*** returns an integer representing the sum of odd elements of the given matrix whose i^{th} and j^{th} index are the same.

The function/method ***calculateMatrixSum*** accepts three arguments - *rows*, an integer representing the number of rows of the given matrix, *columns*, an integer representing the number of columns of the given matrix and *matrix*, representing a two-dimensional array of integers.

The function/method ***calculateMatrixSum*** compiles successfully but fails to return the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
```

Code Analysis

Average-case Time Complexity

© 2026 SHL and/or its affiliates. All rights reserved..

```
{
  int calculateMatrixSum(int rows, int columns, int matrix[][])
  {
    int i, j, sum=0;
    if((rows>0) && (columns>0))
    {
      for(i=0;i<rows;i++)
      {
        for(j=0;j<columns;j++)
        {
          if(i==j)
          {
            if(matrix[i][j]%2!=0)
              sum += matrix[i][j];
          }
        }
      }
      return sum;
    }
    else
      return sum;
  }
}
```

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

8/8

100%	100%	100%
Basic(**4**/4)	Advance(**3**/3)	Edge(**1**/1)

Compilation Statistics

2	2	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:01:33**
Average time taken between two compile attempts:	**00:00:47**
Average test case pass percentage per compile:	**100%**

© 2026 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

The function/method ***allExponent*** returns a real number representing the result of exponentiation of base raised to power exponent for all input values. The function/method ***allExponent*** accepts two arguments - *baseValue*, an integer representing the base and *exponentValue*, an integer representing the exponent.

The incomplete code in the function/method ***allExponent*** works only for positive values of the exponent. You must complete the code and make it work for negative values of exponent as well.

Another function/method ***positiveExponent*** uses an efficient way for exponentiation but accepts only positive *exponent* values. You are supposed to use this function/method to complete the code in ***allExponent*** function/method.

Helper Description

The following class is used to represent a Exponent and is already implemented in the default code (Do not write this definition again in your code):

```
public class Exponent

{

   public int base;

   public int exponent;

   int positiveExponent()

   {

      /*It calculate the Exponent for positive value of exponentValue

       This can be called as -

       Exponent exp = new Exponent(baseValue, exponentValue);
```

© 2026 SHL and/or its affiliates. All rights reserved..

```
        float res = exp.positiveExponent();*/

    }

}
```

Scores

Final Code Submitted — Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4    float allExponent(int baseValue, int exponentValue)
5    {
6       float res = 1;
7       if(exponentValue >=0)
8       {
9          Exponent exp = new Exponent(baseValue, exponentVa
   lue);
10         res = (float)exp.positiveExponent();
11      }
12      else
13      {
14         Exponent exp = new Exponent(baseValue, -exponentV
   alue);
15         res = 1/(float)exp.positiveExponent();
16      }
17      return res;
18   }
19 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

6/6

100%	100%	100%
Basic(**2**/2)	Advance(**3**/3)	Edge(**1**/1)

Compilation Statistics

3	2	1	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:01:59**
Average time taken between two compile attempts:	**00:00:40**
Average test case pass percentage per compile:	**100%**

© 2026 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

The function/method ***productMatrix*** accepts three arguments - *rows,* an integer representing the rows of the matrix; *columns,* an integer representing the columns of the matrix and *matrix*, a two-dimensional array of integers, respectively.

The function/method ***productMatrix*** return an integer representing the product of the odd elements whose i^{th} and j^{th} index are the same. Otherwise, it returns 0.

The function/method ***productMatrix*** compiles successfully but fails to return the desired result for some test cases. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
public class Solution
{
  public int productMatrix(int rows, int columns, int matrix[][])
  {
    int result=1;
    boolean found = false;
    for(int i=0;i<rows;i++){
      for(int j=0;j<columns;j++){
        if((i==j) && (matrix[i][j]%2!=0)){
          result *=matrix[i][j];
          found = true;
        }
      }
    }
    if(found)
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

© 2026 SHL and/or its affiliates. All rights reserved..

```
        return result;
    else
        return 0;

  }
}
```

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

8/8

100%	100%	100%
Basic(**5**/5)	Advance(**2**/2)	Edge(**1**/1)

Compilation Statistics

4	2	2	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:04:16**
Average time taken between two compile attempts:	**00:01:04**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: Java)

You are given predefined structure ***Time*** containing *hour*, *minute*, and *second* as members. A collection of functions/methods for performing some common operations on times is also available. You must make use of these functions/methods to calculate and return the difference.

© 2026 SHL and/or its affiliates. All rights reserved..

The function/method ***difference_in_times*** accepts two arguments - *time1*, and *time2*, representing two times and is supposed to return an integer representing the difference in the number of seconds.

You must complete the code so that it passes all the test cases.

.

Helper Description

The following class is used to represent a Time and is already implemented in the default code (Do not write this definition again in your code):

```
public class Time

{

   public int hour;

   public int minute;

   public int second;

   public Time(int h, int m, int s)

   {

      hour = h;

      minute = m;

      second = s;

   }

   public int compareTo(Object anotherTime)

   {

      /*Return 1, if time1 is greater than time2.

      Return -1 if time1 is less than time2

      or, Return 0, if time1 is equal to time2

      This can be called as -

      *  If time1 and time2 are two Time then -

      *  time1.compareTo(time2) */

   }

   public void addSecond()

   {

      /* Add one second in the time;

      This can be called as -
```

© 2026 SHL and/or its affiliates. All rights reserved..

```
    * If time1 is Time then -

    * time1.addSecond() */

  }
```

Scores

Final Code Submitted — Compilation Status: Fail

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4   int difference_in_times(Time time1, Time time2)
5   {
6     int t1 = time1.hour*3600 + time1.minute*60 + time1.second
7     int t2 = time2.hour*3600 + time2.minute*60 + time2.second;
8     return Math.abs(t1-t2);
9   }
10 }
11
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

```
Solution.java:6: error: ';' expected
int t1 = time1.hour*3600 + time1.minute*60 +
time1.second
^
1 error
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Compilation Statistics

2	0	2	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:02:47**
Average time taken between two compile attempts:	**00:01:24**
Average test case pass percentage per compile:	**0%**

© 2026 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method ***replaceValues*** is modifying the input list in such a way - if the length of input list is odd, then all the elements of the input list are supposed to be replaced by 1s and in case it is even, the elements should be replaced by 0s.

For example: given the input list [0 1 2], the function will modify the input list like [1 1 1]

The function/method ***replaceValues*** accepts two arguments - *size*, an integer representing the size of the given input list and *inputList*, a list of integers representing the input list.

The function/method ***replaceValues*** compiles successfully but fails to get the desired result for some test cases due to incorrect implementation of the function. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
{
  void  replaceValues(int size, int[] inputList)
  {
    int i , j;
    if( size % 2 == 0 )
    {
      for(i = 0; i<size; i++)
      {
        inputList[i] = 0;
      }
    }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

© 2026 SHL and/or its affiliates. All rights reserved..

```
14      else
15      {
16         for(j = 0; j<size; j++)
17         {
18            inputList[j] = 1;
19         }
20      }
21   }
22 }
23
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution Passed TC: **100%**

Total score 8/8

100%	0%	100%
Basic(**6**/6)	Advance(**0**/0)	Edge(**2**/2)

Compilation Statistics

3	3	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:02:42**
Average time taken between two compile attempts:	**00:00:54**
Average test case pass percentage per compile:	**66.7%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: Java)

© 2026 SHL and/or its affiliates. All rights reserved..

The function/method ***sortArray*** modify the input list by sorting its elements in descending order.
The function/method ***sortArray*** accepts two arguments - *len,* representing the length of the list and *arr,* a list of integers representing the input list, respectively.

The function/method ***sortArray*** compiles successfully but fails to get the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
{
  public void sortArray(int len, int[] arr)
  {
    int i , max , location , j , temp;
    for( i = 0 ; i < len ; i ++ )
    {
      max = arr[i];
      location = i;
      for( j = i ; j < len ; j ++ )
      {
        if( arr[j] > max)
        {
          max = arr[j];
          location = j;
        }
      }
      temp = arr[i];
      arr[i] = arr[location];
      arr[location] = temp;
    }
  }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution Passed TC: **100%**

Total score 8/8

100%	100%	100%
Basic(**3**/3)	Advance(**4**/4)	Edge(**1**/1)

© 2026 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

1	1	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:14**
Average time taken between two compile attempts:	**00:00:14**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2026 SHL and/or its affiliates. All rights reserved..

4 Learning Resources

English Comprehension

- Read opinions to improve your comprehension
- Read research papers online
- Read research papers to further enhance your skills

Logical Ability

- Practice inductive reasoning on visual data
- Learn about pattern identification
- Practice your Inductive Reasoning Skills!

Quantitative Ability (Advanced)

- Learn about the real life applications of probability
- Learn about the real life applications of logarithms
- Learn about the application of Bayes' Theorem in varied fields

Icon Index

Free Tutorial	Paid Tutorial	Youtube Video	Web Source
Wikipedia	Text Tutorial	Video Tutorial	Google Playstore

© 2026 SHL and/or its affiliates. All rights reserved..